- **对象声明语法**

```
let 对象名 = {}
```

  - 例如：

```
// 声明了一个person对象
let person = {}
// 检测person变量的数据类型
console.log(typeof person)// object
```

- **对象有属性和方法组成**
  - 属性：信息或叫特征（名词）。 比如 手机尺寸、颜色、重量等...
  - 方法：功能或叫行为（动词）。 比如 手机打电话、发短信、玩游戏...

```
let 对象名 = {
    属性名: 属性值,
    方法名: 函数
}
```

- **属性**
  - 数据描述性的信息称为属性，如人的姓名、身高、年龄、性别等，一般是名词性的。

```
let person = {
    uname: 'andy',
    age: 18,
    sex: '男'
}
```

  - 属性都是成 对出现的，包括属性名和值，它们之间使用英文 : 分隔
  - 多个属性之间使用英文 , 分隔
  - 属性就是依附在对象上的变量（外面是变量，对象内是属性）
  - 属性名可以使用 "" 或 ''，一般情况下省略，除非名称遇到特殊符号如空格、中横线等

- **属性访问**
  - 声明对象，并添加了若干属性后，可以使用 . 或 [] 获得对象中属性对应的值，我称之为属性访问。简单理解就是获得对象里面的属性值。

```
let person = {
    name: 'andy',
    age: 18,
    sex: '男'
}
console.log(person.name)
console.log(person.age)
```

或者

```
let person = {
    name: 'andy',
    age: 18,
    sex: '男'
}
console.log(person['name'])
console.log(person['age'])
```

- 两种情况只能用[]：
  - 当属性名是变量的时候
  - 当属性名有特殊字符的时候

- **属性批量访问**
  - 格式：

```
let goods = {
    name: '小米10',
    num: 100012816024,
    weight: '0.55kg',
    address: '中国大陆'
}
for(let key in goods){
    console.log(goods[key]) // 正确的
    // console.log(goods['key']) // 错误的，代码不报错，取出来的值是 undefined
    // console.log(goods.key)// 错误的，代码不报错，取出来的值是 undefined
}
```

- 这个对象有几个属性和函数就循环几次，每次循环后key的值变成属性名或函数名，类型为字符串。
- 如果对象里既有属性又有函数，可以加判断语句，遇到属性就取值，遇到函数就调用。

- **新增属性**
  - 也可以动态为对象添加属性，动态添加与直接定义是一样的，只是语法上更灵活。

```
let person = {
    name: 'andy',
    age: 18,
}
person.hobby = '足球'
person['sex'] = '男'
console.log(person)
```

- **修改属性**

```
let mobile = {
  brand: '小米',
  price: 3980
}
// 把价格改为3600，下面两种方式等价
mobile.price = 3600
mobile['price'] = 3600
```

```
Elements  Console  Sources  Performance
top ▼  Filter
{brand: "小米", price: 3600}
```

- **删除属性**

```
let mobile = {
  brand: '小米',
  price: 3980,
  type: 'CC9 Pro'
}
// 删除type属性，下面两种方式等价
delete mobile.type
delete mobile['type']
```

```
Elements  Console  Sources  Performance
top ▼  Filter
{brand: "小米", price: 3600}
```

- **对象中的方法**
  - 数据行为性的信息称为方法，如跑步、唱歌等，一般是动词性的，其本质是函数。

```
let person = {
    name: 'andy',
    sayHi: function() {
        document.write('hi~~')
    }
}
```

  - 方法是由方法名和函数两部分构成，它们之间使用：分隔
  - 多个属性之间使用英文，分隔
  - 方法是依附在对象中的函数
  - 方法名可以使用 "" 或 ''，一般情况下省略，除非名称遇到特殊符号如空格、中横线等

- **对象中的方法访问**
  - 声明对象，并添加了若干方法后，可以使用 . 调用对象中函数，我称之为方法调用。

```
let person = {
    name: 'andy',
    sayHi: function () {
        document.write('hi~~')
    }
}
// 对象名.方法名()
person.sayHi()
```

- 注意： 千万别忘了给方法名后面加小括号

- **新增对象中的方法**
  - 也可以动态为对象添加方法，动态添加与直接定义是一样的，只是语法上更灵活

```
person.move = function() {
    document.write('移动一点点')
}
```

  - 注：无论是属性或是方法，同一个对象中出现名称一样的，后面的会覆盖前面的。

- **null**
  - null 也是 JavaScript 中数据的一种，类型为对象，通常只用它来表示不存在的对象。使用 typeof 检测类型它的类型时，结果为 object。

- **内置对象是什么**
  - JavaScript内部提供的对象，包含各种属性和方法给开发者调用
  - 思考：我们之前用过内置对象吗?
    - document.write()
    - console.log()

- **内置对象Math**
  - Math对象是JavaScript提供的一个“数学高手”对象
  - 提供了一系列做数学运算的方法
  - 方法有：
    - random：生成0-1之间的随机数（包含0不包括1）
    - ceil：向上取整
    - floor：向下取整
    - max：找最大数
    - min：找最小数
    - pow：幂运算
    - abs：绝对值

- **生成任意范围随机数**
  - 如何生成0-10的随机数呢?

```
Math.floor(Math.random() * (10 + 1))
```

  - 如何生成5-10的随机数?

```
Math.floor(Math.random() * (5 + 1)) + 5
```

- 如何生成N-M之间的随机数

```
Math.floor(Math.random() * (M - N + 1)) + N
```

- **对象的key的顺序**
  - key是数字
    - 按照数字大小排序

```
const data = {
  3: 1,
  2: 1,
  1: 1
}

console.log(Object.keys(data)) // ['1', '2', '3']
```

  - key是字符串
    - 按照插入顺序排序

```
const data = {
  姓名: 1,
  年龄: 1,
  性别: 1
}

console.log(Object.keys(data)) // ['姓名', '年龄', '性别']
```

  - key同时包含数字和字符串
    - 先按照【数字 -> 字符串】的顺序排序，再按数字和字符串各自的排序方式排序

```
const data = {
  性别: 1,
  2: 1,
  年龄: 1,
  3: 1,
  姓名: 1,
```

```
  1: 1
}

console.log(Object.keys(data)) // ['1', '2', '3', '性别', '年龄', '姓名']
```

- 参考：
  - Property order is predictable in JavaScript objects since ES2015
  - The traversal order of object properties in ES6